PostgreSQL + pgAdmin

Instalación, configuración y creación de la BBDD para el proyecto Alambres Ramiro

Acceso a Datos · CFGS Desarrollo de Aplicaciones Multimedia

Docente: José Manuel Villar Ferradal

Objetivos de la práctica

Instalar PostgreSQL y pgAdmin 4 en Windows, Linux o macOS.

Distinguir servidor PostgreSQL, cliente pgAdmin y aplicación Java.

Crear la base de datos tienda_ramiro y un usuario específico para la aplicación.

Crear las tablas cliente, producto y movimiento_stock.

Insertar datos de prueba y verificar la instalación con consultas SQL.

Dejar el entorno preparado para el siguiente paso: conexión JDBC desde Java.

¿Qué estamos instalando?

Tres piezas diferentes que colaborarán durante la unidad



PostgreSQL almacena y gestiona los datos.

pgAdmin permite administrar PostgreSQL y ejecutar SQL visualmente.

Java se conectará después al mismo servidor mediante JDBC.

Configuración común para toda la clase

```
Servidor: localhost
  Puerto:   5432
BBDD:     tienda_ramiro
 Admin:    postgres
App:      tienda_ramiro
```

5432 es el puerto habitual de PostgreSQL.

postgres será el usuario administrador durante la preparación.

tienda_ramiro será el usuario utilizado por la aplicación Java.

La contraseña de clase es didáctica: en proyectos reales no se guarda en el código.

Instalación en Windows

1. Entrar en la página oficial de descargas de PostgreSQL.
2. Descargar el instalador de Windows y ejecutarlo.
3. Mantener PostgreSQL Server, pgAdmin 4 y Command Line Tools.
4. Elegir una contraseña para el usuario administrador postgres.
5. Mantener el puerto 5432.
6. Finalizar la instalación y abrir pgAdmin 4.
7. Comprobar que el servicio PostgreSQL está iniciado.

Comprobación opcional en PowerShell:

```
psql --version
```

Instalación en Linux · Ubuntu / Debian

```
sudo apt update

sudo apt install postgresql postgresql-contrib

sudo systemctl status postgresql

sudo -u postgres psql

SELECT version();

\q
```

apt instala PostgreSQL desde los paquetes de la distribución.

systemctl permite comprobar el estado del servicio.

sudo -u postgres psql abre el cliente de terminal como administrador.

Dentro de psql, \q vuelve a la terminal de Linux.

pgAdmin 4 Desktop puede instalarse después desde el repositorio oficial de pgAdmin.

Instalación en macOS

1. Descargar PostgreSQL desde la página oficial para macOS.
2. Instalar el servidor PostgreSQL.
3. Definir la contraseña del usuario postgres.
4. Mantener el puerto 5432.
5. Descargar pgAdmin 4 para macOS.
6. Elegir el paquete adecuado: Apple Silicon (arm64) o Intel (x64).
7. Abrir pgAdmin y comprobar la conexión local.

Comprobación en Terminal:

```
psql --version
```

Primera conexión desde pgAdmin 4

```
Name:                  PostgreSQL local
    Host name/address:    localhost
       Port:                 5432
     Maintenance database: postgres
     Username:             postgres
     Password:             ********
```

La contraseña maestra de pgAdmin no es necesariamente la contraseña de PostgreSQL.

Una vez conectado veremos Databases, Login/Group Roles y Tablespaces.

Abriremos Tools → Query Tool para ejecutar nuestras primeras sentencias SQL.

Primera comprobación SQL

```
SELECT version();

SELECT current_database();

SELECT current_user;
```

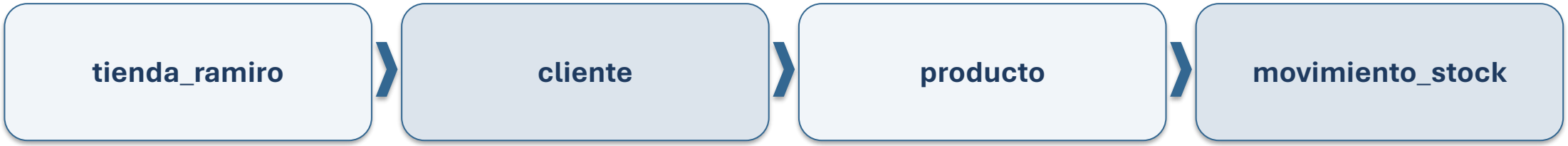
version(): versión del servidor PostgreSQL.

current_database(): base de datos actual.

current_user: usuario con el que estamos conectados.

Antes de continuar, los tres resultados deben tener sentido.

Base de datos del proyecto JDBC



cliente y producto corresponden directamente con las clases Java del proyecto.

movimiento_stock permitirá practicar relaciones y, más adelante, transacciones.

Usaremos claves primarias generadas automáticamente y restricciones de integridad.

Paso 1 · Crear usuario y base de datos

Ejecutar como administrador postgres

```
CREATE USER tienda_app
WITH PASSWORD 'tienda1234';

CREATE DATABASE tienda_ramiro;

GRANT CONNECT
ON DATABASE tienda_ramiro
TO tienda_ramiro;
```

No conectaremos Java como superusuario postgres.

tienda_ramiro tendrá únicamente los permisos que necesite.

Después de CREATE DATABASE, cambiaremos Query Tool para trabajar sobre tienda_ramiro.

Paso 2 · Entrar en tienda_ud2 y preparar el esquema

```
SELECT current_database();

   GRANT USAGE
ON SCHEMA public
TO tienda_ramiro;

  GRANT CREATE
ON SCHEMA public
TO tienda_ramiro;
```

current_database() debe devolver tienda_ramiro.

USAGE permite utilizar los objetos del esquema.

CREATE permitirá crear objetos durante las prácticas si lo necesitamos.

A partir de aquí, las tablas se crean dentro de tienda_ramiro.

Paso 3 · Crear la tabla cliente

```
CREATE TABLE cliente (
id INTEGER GENERATED ALWAYS AS IDENTITY,
nombre VARCHAR(100) NOT NULL,
email VARCHAR(150) NOT NULL,
activo BOOLEAN NOT NULL DEFAULT TRUE,

CONSTRAINT pk_cliente PRIMARY KEY (id),
CONSTRAINT uq_cliente_email UNIQUE (email)
);
```

IDENTITY genera automáticamente el id.

NOT NULL obliga a proporcionar el dato.

UNIQUE impide repetir el email.

DEFAULT TRUE asigna un valor cuando no se especifica activo.

Paso 4 · Crear la tabla producto

```
CREATE TABLE producto (
id INTEGER GENERATED ALWAYS AS IDENTITY,
nombre VARCHAR(120) NOT NULL,
precio NUMERIC(10,2) NOT NULL,
stock INTEGER NOT NULL DEFAULT 0,

CONSTRAINT pk_producto PRIMARY KEY (id),
CONSTRAINT ck_producto_precio CHECK (precio >= 0),
CONSTRAINT ck_producto_stock CHECK (stock >= 0)
);
```

NUMERIC(10,2) es apropiado para importes.

CHECK protege la integridad incluso si Java envía un dato incorrecto.

La base de datos también valida: no toda la responsabilidad pertenece a la aplicación.

Paso 5 · Crear movimiento_stock

```
CREATE TABLE movimiento_stock (
id BIGINT GENERATED ALWAYS AS IDENTITY,
producto_id INTEGER NOT NULL,
cantidad INTEGER NOT NULL,
fecha TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,

CONSTRAINT pk_movimiento_stock PRIMARY KEY (id),

CONSTRAINT fk_movimiento_producto
FOREIGN KEY (producto_id)
REFERENCES producto(id)
);
```

producto_id es una clave foránea.

Cada movimiento debe apuntar a un producto existente.

CURRENT_TIMESTAMP registra automáticamente la fecha.

Esta tabla será útil para practicar COMMIT y ROLLBACK.

Paso 6 · Permisos para la aplicación

```
GRANT SELECT, INSERT, UPDATE, DELETE
   ON ALL TABLES IN SCHEMA public
         TO tienda_ramiro;

        GRANT USAGE, SELECT
  ON ALL SEQUENCES IN SCHEMA public
         TO tienda_ramiro;
```

SELECT: consultar.

INSERT: crear filas.

UPDATE: modificar.

DELETE: eliminar.

Las secuencias/identidades necesitan permisos para generar claves.

Paso 7 · Cargar datos de prueba

```
INSERT INTO cliente(nombre,email,activo)
                VALUES
   ('Ana Ruiz','ana@ejemplo.com',TRUE),
 ('Marta López','marta@ejemplo.com',TRUE),
('Pedro García','pedro@ejemplo.com',FALSE);

INSERT INTO producto(nombre,precio,stock)
                VALUES
     ('Teclado mecánico',59.90,10),
     ('Ratón inalámbrico',24.95,25),
    ('Monitor 27 pulgadas',229.99,8),
     ('Auriculares USB',39.50,15);
```

No indicamos id: PostgreSQL lo genera.

Estos datos permiten empezar inmediatamente con SELECT y ResultSet.

Después los INSERT se ejecutarán desde Java con PreparedStatement.

Paso 8 · Comprobar SELECT, UPDATE y DELETE

```
SELECT * FROM cliente ORDER BY id;
SELECT * FROM producto ORDER BY id;

UPDATE producto
SET precio = 54.90
WHERE id = 1;

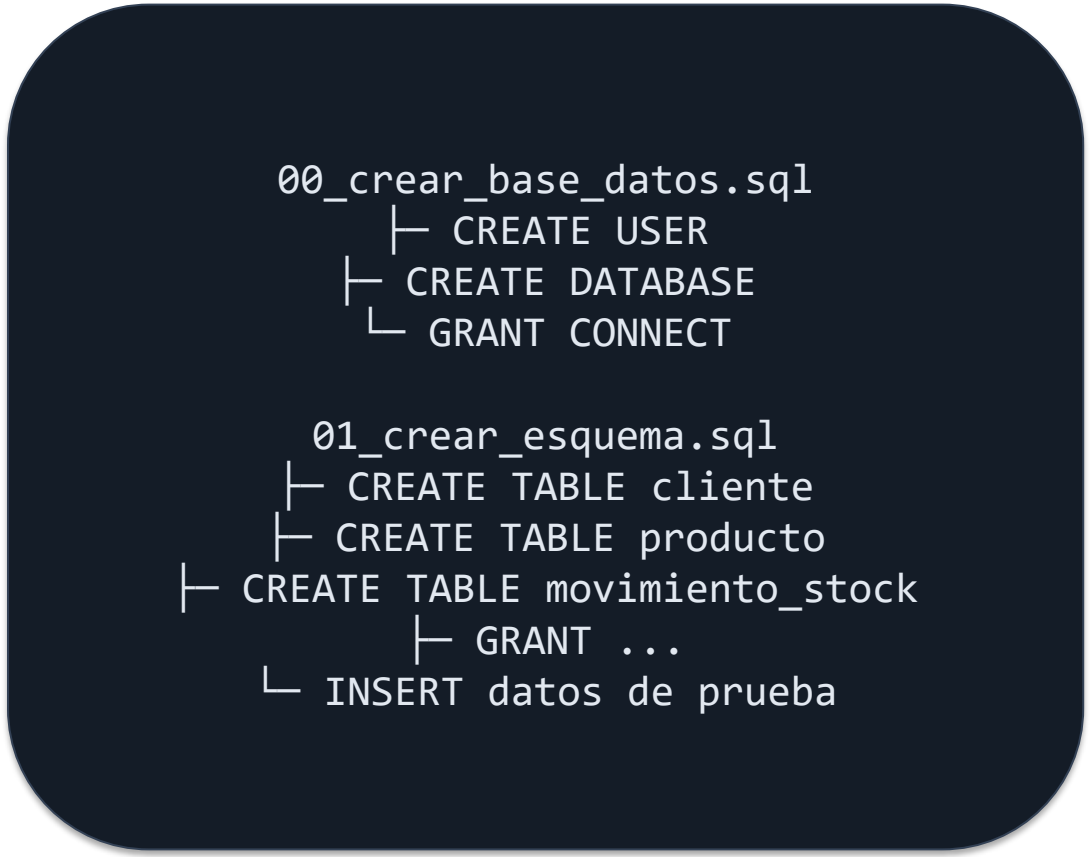
DELETE FROM cliente
WHERE email = 'temporal@email.com';
```

Antes de programar JDBC debemos saber ejecutar SQL directamente.

Java enviará posteriormente estas mismas clases de sentencias.

CRUD: Create → INSERT · Read → SELECT · Update → UPDATE · Delete → DELETE.

Organización recomendada de los scripts



El script 00 se ejecuta conectado como administrador.

Después cambiamos a tienda_ud2.

El script 01 crea la estructura de trabajo.

Separarlos ayuda a entender servidor, base de datos, esquema y tablas.

Problemas frecuentes

pgAdmin no conecta → comprobar que el servicio PostgreSQL está iniciado.

Connection refused → revisar host y puerto 5432.

Password authentication failed → revisar usuario y contraseña.

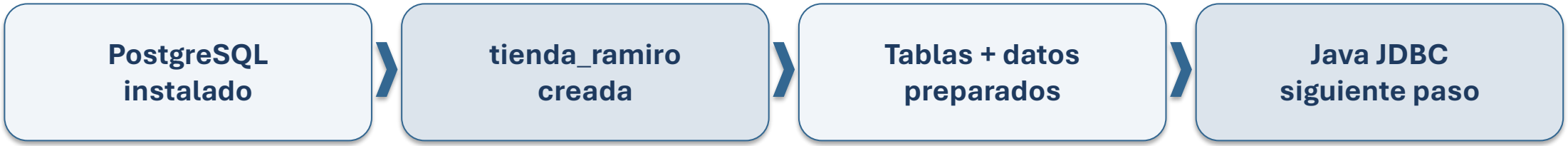
relation … does not exist → comprobar que estamos en tienda_ud2 y en el esquema correcto.

permission denied → revisar GRANT para tienda_app.

La tabla no aparece en pgAdmin → refrescar Schemas → public → Tables.

Punto de llegada

Cuando esto funcione, estamos preparados para JDBC




```
jdbc:postgresql://localhost:5432/tienda_ud2

usuario: tienda_app
contraseña: tienda1234
```